

GitScout — Software Project Management Plan (SPMP)

Group 7

Submission Date: February 17, 2026

1. Introduction

This Software Project Management Plan (SPMP) describes how the GitScout project will be organized, planned, executed, and monitored. The plan defines team structure, project management tools, risk management strategies, estimation approach, scheduling framework, and documentation practices. This SPMP complements the Software Configuration Management Plan (SCMP) and is maintained under version control in the GitHub repository.

2. Project Overview

GitScout is a web-based talent sourcing platform that analyzes publicly available GitHub profile and activity data to help recruiters and hiring managers identify and evaluate developer candidates. The Minimum Viable Product (MVP) includes advanced search and filtering, candidate analytics dashboards, a transparent and versioned talent scoring algorithm (v1), candidate shortlists, and export functionality (CSV/PDF).

3. Software Development Process Model

GitScout follows an Agile iterative development lifecycle. Work is organized into short sprints that include planning, implementation, integration, testing, review, and release activities. Each sprint delivers a functional increment of the system, allowing the team to continuously refine features, address feedback, and maintain alignment with project goals.

Development tasks originate as GitHub issues, are implemented through pull requests, and are validated through automated checks and team review before merging into the main branch. This iterative process helps the team track progress, review changes, and improve the system across iterations throughout the project lifecycle.

4. Organization and Roles

- Project Manager: Xiang He — Responsible for overall project coordination, sprint planning, risk tracking, backlog prioritization, milestone baselines, and release readiness approval.
- Frontend Developers: Khushi Sridhar, Haomiao Hu — Responsible for UI implementation, component standards, frontend architecture, usability consistency, and frontend test coverage.

- Backend Developers: Bowang Lang, Cong Chen — Responsible for API design, database schema management, scoring logic implementation, backend integration, and backend test coverage.
- UX/UI Designer: Haomiao Hu — Responsible for UI flows, wireframes, information hierarchy, usability validation, and design artifact maintenance.
- QA/Testing Lead: Joseph Ma — Responsible for defining the test strategy, validating CI checks, maintaining quality gates, and verifying release readiness.

5. Project Management Tools

These tools help the team coordinate work, track progress, and maintain consistency across development activities.

- Version Control: Git and GitHub with protected main branch and pull request workflow.
- Issue Tracking and Backlog Management: GitHub Issues and GitHub Project Board.
- Continuous Integration (CI): GitHub Actions for linting, type-checking, unit tests, and smoke tests.
- Development Stack: Next.js (frontend), Elysia/Bun (backend), TypeScript.
- Documentation: Markdown documents stored under `/docs` and `/design-docs` in the repository.
- Communication: Scheduled stand-ups (2–3 times per week), sprint planning, demos, and retrospectives.

6. Risk Management

Risks are tracked as GitHub Issues labeled 'risk:*' with assigned owners and mitigation plans. Each risk is discussed by the team, prioritized informally based on impact, and handled through practical mitigation steps.

- Key Identified Risks:

GitHub API rate limiting: mitigation includes caching, backoff strategies, and degraded-mode handling.

Schema migration errors: mitigation includes migration per PR, review, CI smoke tests, rollback runbook.

Uncontrolled merges: mitigation includes protected branches, mandatory PR reviews, CI gates.

Quality regressions: mitigation includes automated tests, regression testing, release checklist.

7. Estimation and Scheduling

Work is planned using a backlog of user stories and organized into two-week sprints. Tasks are estimated based on team discussion. Each sprint aims to deliver a working increment.

8. Work Breakdown Structure

The GitScout project work is organized into major functional categories aligned with the Agile development lifecycle. These include requirements analysis, UI/UX design, backend

development, frontend development, system integration, testing, and deployment. Tasks from each category are tracked as GitHub Issues and grouped into sprints based on priority and dependencies.

9. Monitoring and Reporting

Project progress is monitored through GitHub Project boards, sprint demos, and retrospectives. Stand-up meetings track progress, upcoming tasks, and project risks.

Release readiness requires passing CI checks, updated documentation, and tagged versions. The SPMP is maintained as a controlled document and updated through pull requests as project planning evolves. Progress is tracked using GitHub issues, pull requests, and sprint reviews to ensure steady development and feature completion.

10. GitHub Repository

GitHub Group Project Repository:

<https://github.com/xors-software/git-scout>

11. Team Member Contributions Since Last Submission

Xiangnan He: Aligned the team on sprint structure and Agile workflow, converted requirements into GitHub user stories, established SCM and CI policies, and coordinated early frontend-backend integration planning.

Khushi Sridhar: Reviewed frontend requirements, evaluated UI libraries, and outlined the main interface structure in alignment with repository organization and SCM practices. Implemented the frontend API client and reusable hooks for candidate search and shortlist features, preparing the groundwork for upcoming UI development.

PR link: <https://github.com/xors-software/git-scout/pull/16>

Bowang Lang: Implemented the search API endpoint with /search prefix. Developed GET /search/candidates with query filters, sorting, and pagination. Integrated GitHub search service and scoring service, returned paginated results, and registered the route in server/src/index.ts.

PR link: <https://github.com/xors-software/git-scout/pull/20>

Cong Chen: Implemented the backend SQLite database layer, including schema design for shortlist and profile_cache tables. Created reusable database helper functions and integrated the data layer with API routes. Tested end-to-end functionality via Swagger and pushed the feature branch for review.

PR link: <https://github.com/xors-software/git-scout/pull/18>

Haomiao Hu: Defined UI/UX principles, documented user interaction flows, and ensured interface design consistency with project requirements and SCM practices.

Joseph Ma: Standardized development and testing environments, configured Git and Node versions using NVM, and validated local test execution to support reliable CI integration. Added initial server-side smoke/integration tests using bun+elysia. Finished health endpoint search endpoint and candidates endpoints. Also minor server bootstraps adjustment to allow importing app in tests. Clean up starters artifacts.

PR links: <https://github.com/xors-software/git-scout/pull/17>

<https://github.com/xors-software/git-scout/pull/15>